

Technische Universität Dresden

Faculty of Mathematics

Scientific Research and Writing - Neural Networks, Deep Learning and
Applications

Classifying braking data and artificial data generation - FSD Project

Group 1 - Gemini

Members:

- Sara Dalmau Guamis
- Anupama Chandrashekhar Hegde
- Oleksandr Malakhatka
- Linus Röber

June 2025

Contents

1	Introduction and Data Overview	3
1.1	Introduction	3
1.2	Data overview	3
2	Modeling	4
2.1	FCN	4
2.1.1	Methods	4
2.1.2	Results and Analysis	5
2.2	2D CNN	7
2.2.1	Experimental Setup	7
2.2.2	Key Observations	8
2.2.3	Chosen Models	9
2.3	LSTM	12
2.3.1	Methodology	12
2.3.2	Results and Analysis	14
2.4	Additional Models	16
2.4.1	Transformer Models	16
2.4.2	XGBoost with Classifier Chains	17
2.5	Best Model	19
3	Generative AI	21
3.1	Visualization	21
3.1.1	Mathematical Foundation	21
3.1.2	Results	22
3.2	GAN	23
3.2.1	Theory	23
3.2.2	Methodology	23
3.2.3	Results	23
3.3	RNN	25
3.3.1	Methodology	25
3.3.2	Results	25
3.4	Comparison	25
3.5	Extending the dataset	26
3.5.1	Methodology	26
3.5.2	Results	26

4	Enseble Models	29
4.1	Motivation	29
4.2	Candidate Ensemble Techniques	29
4.3	Results	29
5	Conclusion	32

Chapter 1

Introduction and Data Overview

1.1 Introduction

FSD provides the project task as two data frames along with a multi-classification task. The main aim is to develop a model that can effectively predict if any of the car’s brakes don’t comply with German road and car safety standards, and potentially as part of the methodology for car safety inspection vendors.

Data readings provided with proprietary diagnostic FSD device which consists of real faulty breaks and simulated tests.

4 people conduct the project, and the report is organized so that each person is assigned sections and/or chapters, which is indicated by color.

1.2 Data overview

Data consists of sensor readings of shape 128×6 per item, 128 time points with values of deceleration and angular deceleration in 3 axes and tabular data: velocity, mass, deceleration average, and model. Two data frames were provided, train and test. Upon further investigation, it was noticed that the test dataset is greatly biased compared to the training dataset.

“Model” is a categorical variable that has no common categories across the two datasets and therefore has no practical use.

“Label” is presented as 12 classes in the train dataset and four classes in the test, that encode a Boolean vector of length 4, with **True** value corresponding to an incompliant break on one of the sides. It was considered to experiment with a multi-label task along with a multi-class task. This represents the actual task more naturally, without hiding the inter-correlation between classes, and helps to extrapolate the model to “16 classes” which naturally arise instead of the 12 provided.

Normalization of data was considered a standard practice with Neural Networks, to avoid gradient explosions and potentially make training performance better.

Topological data analysis was used for feature engineering, but gave no substantial performance boost comparable to a lengthy explanation of the theory behind it, so method will be described rather shortly. A *persistence image* condenses multi-scale shape information in a data set into a fixed-length list of numbers: it turns each “feature lifetime” point into a blurred heat-map, samples that map on a regular grid, and reads off the resulting values so they can be used directly as standard machine-learning features.[1]

Chapter 2

Modeling

2.1 FCN

2.1.1 Methods

Data Used and Preprocessing

All the datasets used in this work (`X_train`, `y_train`, `X_test`, `y_test`, `X_ext_train`, `X_ext_test`) were obtained from the previous preprocessing step done by the group. This preprocessing included normalization of all features and handling of missing values.

During data inspection, we found missing values (NaN) in the mass column of `X_test` and `X_ext_test`. Since all columns were normalized in the preprocessing step, the mean of the mass column in `X_train` is 0. Therefore, we replaced missing values in mass with 0 in both test datasets. This avoids introducing bias, as 0 represents the mean after normalization.

Model Variants and Data Types

In this project, we explored different neural network architectures and training strategies to classify deceleration measurement sequences. We worked with two main types of data: the original data and the extended data, each leading to different model designs and input features.

Original Data For the original data, we only used the `sensor_data` column from the DataFrame. This column contains time series data for each sample: 128 time points for 6 channels (three linear accelerations and three angular velocities). The other tabular features (such as mass, velocity, deceleration average, and model) were not used in these models.

We implemented a **Fully Convolutional Network (FCN1D)** for this data. The FCN1D model is designed for time series classification and consists of three 1D convolutional layers with batch normalization and ReLU activation, followed by adaptive average pooling and a final linear layer for classification. The model takes as input the six sensor channels and outputs the predicted class.

For each data type, we tested the following four model variants:

- No Dropout, No Class Weights: Basic model, trained with standard cross-entropy loss and no dropout layers.

- With Dropout, No Class Weights: Dropout layers are added after the convolutional layers to reduce overfitting, but the loss function is not weighted.
- No Dropout, With Class Weights: No dropout is used, but the cross-entropy loss is weighted according to the class distribution to address class imbalance.
- With Dropout, With Class Weights: Both dropout and class weighting are used together.

Extended Data For the extended data, we used all columns available in the DataFrame. This means that, in addition to the `sensor_data` time series, we also included tabular features such as `velocity`, `mass`, `deceleration_average`, as well as topological features (`pers_image`) and the first ten principal components (`PCA_0` to `PCA_9`). All these features were concatenated for each sample, resulting in a larger input vector.

For this data, we implemented a **Fully Connected Deep (FCD)** neural network, which is a feedforward network with two hidden layers, batch normalization, ReLU activation, and optional dropout. The same four model variants described above were tested with the extended data.

Training and Validation

We used random split validation on the training sets (`X_train`, `y_train` for the original data and `X_ext_train`, `y_train` for the extended data) to create validation sets, as required by the assignment. For each model variant and data type, we performed a grid search over hyperparameters such as learning rate, batch size, and dropout probability. Specifically, we tested learning rates of $\{1 \times 10^{-3}, 5 \times 10^{-4}\}$, batch sizes of $\{32, 64\}$, and dropout probabilities of $\{0.2, 0.3, 0.5\}$ (for models with dropout). The Adam optimizer and CrossEntropyLoss were used for training. For models with class weighting, the class weights were computed as the inverse of the class frequencies in the training set. The best model for each configuration was selected based on validation accuracy.

After training, we evaluated the best models on the corresponding test sets (`X_test`, `y_test` for the original data and `X_ext_test`, `y_test` for the extended data) to check for overfitting and to report the final accuracy, as requested in the project instructions.

2.1.2 Results and Analysis

FCN1D Results (Original Data)

Table 2.1 summarizes the best test accuracy for each FCN1D variant. The best accuracy is achieved by the model with dropout and no class weights (46.45%), followed closely by the basic model (44.46%). However, all models show low balanced accuracy (below 0.25) and macro F1 (below 0.13), indicating poor performance on minority classes.

The confusion matrices for the four FCN1D models (see Figure 2.1) show that most predictions are concentrated in class 0, with very few correct predictions for the other classes. For example, in the best model, class 0 is predicted correctly for most samples, but classes 1, 2, 4, and 5 are rarely predicted, and when they are, the recall and precision are very low. This is confirmed by the classification reports in the code, where the recall for minority classes is often below 0.1, and the F1-scores are close to zero.

Table 2.1: Summary of model performance and training times. Column names: A = Dropout, B = Class Weights, C = Best Accuracy, D = Balanced Accuracy, E = Macro F1, F = Weighted F1, G = Learning Rate, H = Batch Size, I = Dropout p, J = Training Time (min). For extended data models, training time per model is estimated by dividing the total time equally among the four variants.

Model	A	B	C	D	E	F	G	H	I	J
FCN1D	No	No	44.46%	0.23	0.12	0.45	0.0005	64	-	1.53
	Yes	No	46.45%	0.24	0.13	0.47	0.0005	32	0.3	3.83
	No	Yes	32.37%	0.18	0.09	0.40	0.001	32	-	1.27
	Yes	Yes	29.57%	0.17	0.10	0.39	0.001	32	0.2	3.91
FCD	No	No	59.24%	0.20	0.15	0.44	0.001	32	-	0.48
	No	Yes	59.24%	0.20	0.15	0.44	0.001	32	-	0.48
	Yes	No	59.24%	0.20	0.15	0.44	0.001	32	0.3	0.48
	Yes	Yes	59.24%	0.20	0.15	0.44	0.001	32	0.3	0.48

Adding class weights or dropout does not significantly improve the results. In fact, models with class weighting tend to have even lower accuracy and F1, suggesting that the imbalance is too severe for simple reweighting to help.

FCD Results (Extended Data)

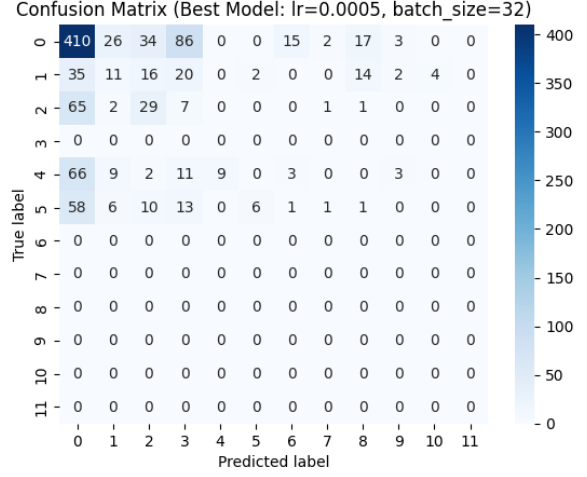
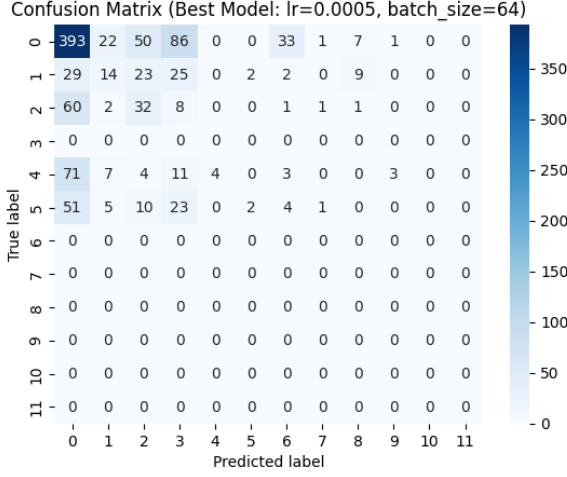
Table 2.1 also shows the results for the FCD models trained on the extended data. All variants reach the same test accuracy (59.24%), which at first glance seems better than the FCN1D models. However, the confusion matrices for these models (see Figure 2.2) reveal that the model simply predicts class 0 for all samples. This is also reflected in the classification reports in the code, where the recall for class 0 is 1.0 and for all other classes is 0.0. The macro F1 and balanced accuracy are very low (0.15 and 0.20, respectively), confirming that the model is not actually learning to distinguish between classes.

Comparison and Recommendation

Although the FCD models achieve higher accuracy, this is only because they always predict the majority class. The FCN1D models, despite their low overall accuracy, at least attempt to predict minority classes, as seen in the confusion matrices and F1-scores. However, none of the models tested provide satisfactory performance for this highly imbalanced multiclass problem.

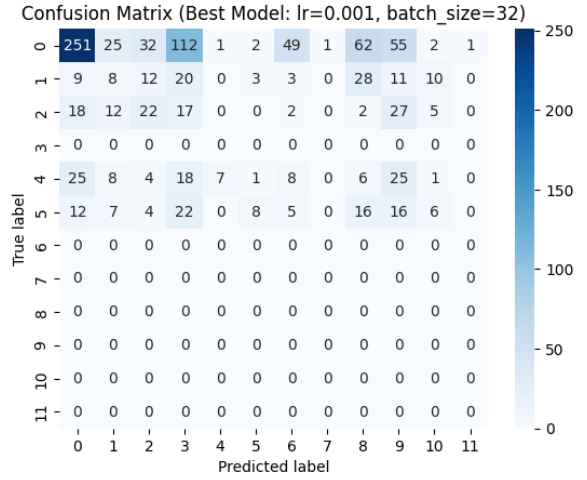
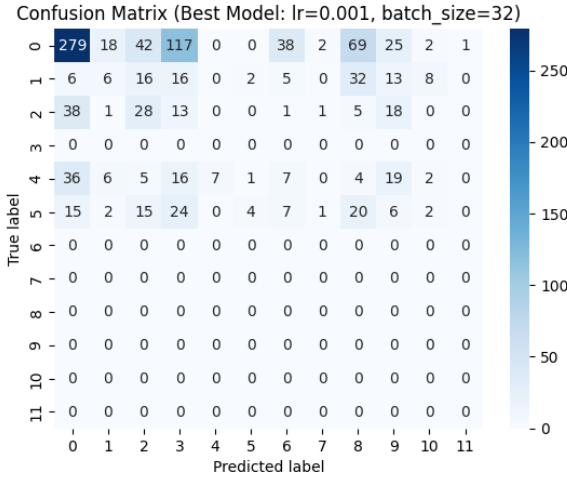
Training times for all models are reported in Table 2.1. The differences in training time between models are relatively small and do not represent a decisive factor when choosing the best model.

Based on these results, the FCN1D with dropout and no class weights is the best among the tested models, as it achieves the highest accuracy and slightly better macro F1. However, the overall performance is still poor, and the model is not suitable for real-world deployment in its current form. The main limitation is the extreme class imbalance, which standard neural network techniques (dropout, class weighting) cannot overcome.



(a) No Dropout, No Weights

(b) Dropout, No Weights



(c) No Dropout, With Weights

(d) Dropout, With Weights

Figure 2.1: Confusion matrices for the four FCN1D models (original data)

2.2 2D CNN

2.2.1 Experimental Setup

Basics

Four model families were evaluated:

- **CNN:** Simple 2-D CNN (unweighted)
- **w-CNN:** Weighted 2-D CNN (Weighted sampler with DVBAE)
- **w-ResNet:** Weighted ResNet (Weighted sampler with DVBAE)
- **CNN-MC:** Simple 2-D CNN (multiclass)

with different configurations tried in each family.

The multi-class case wasn't giving any significant results and would have required additional steps to cross-compare with multi-label architectures, so omitted from further discussion.

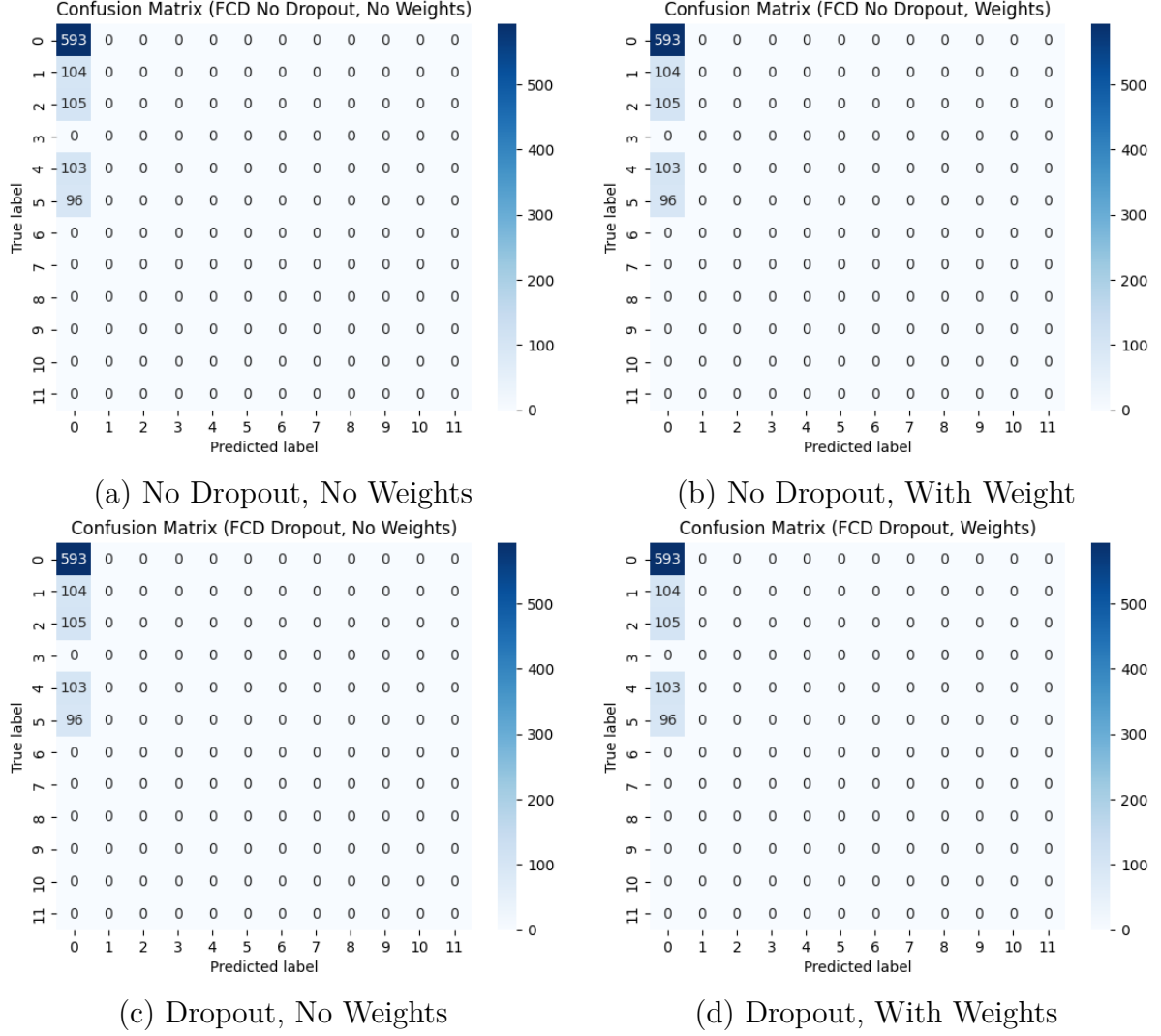


Figure 2.2: Confusion matrices for the four FCN1D models (original data)

Training Loss: Focal/ Normal Binary Cross-Entropy for multi-label, Cross-Entropy for multiclass

$$\mathcal{L}_{\text{FocalBCE}}(p, y) = -y(1-p)^{\gamma} \ln(p) - (1-y)p^{\gamma} \ln(1-p) \quad (2.1)$$

For the so-called weighted models, DBVAE [2] was used.

Data

Model architectures were tested on standard, extended, and persistence datasets. The first two apply CNN over the initial sensor data, and the extended one has features generated based on the prescience images through PCA and the persistence dataset provides persistence images based on zero and first-order homology in 2 channels.

2.2.2 Key Observations

Overall separability. Even in the best case (*Extended*), the vanilla CNN only reaches $\text{AUC} \approx 0.66$ and $\text{ACC} \approx 0.56$. All other cases hover around $\text{AUC} = 0.58\text{--}0.65$ and $\text{acc} = 0.47\text{--}0.61$ —well below what you’d want for a good anomaly detector.

Architecture	Case	micro-F1	macro-F1	binary ACC	binary AUC
CNN	Standard 1	0.31	0.34	0.52	0.58
	Standard 2	0.28	0.32	0.47	0.59
	Extended 1	0.29	0.31	0.56	0.66
	Persistence 1	0.16	0.16	0.48	0.48
w-CNN	Standard 1	0.31	0.34	0.52	0.58
	Standard 2	0.28	0.29	0.49	0.65
	Extended 1	0.29	0.30	0.60	0.62
	Extended 2	0.28	0.31	0.47	0.59
	Extended 3	0.31	0.32	0.61	0.63
	Extended 4	0.28	0.31	0.58	0.64
	Persistence 1	0.12	0.13	0.49	0.48
w-ResNet	Standard 1	0.23	0.24	0.57	0.61

Table 2.2: Test scores of CNNs by architecture and case.

- **Weighted CNN is most consistent.**
 - Best binary accuracy: 0.61 (*Extended 3*).
 - Best AUC: 0.65 (*Standard 2*); sustains 0.62–0.64 in the *Extended* variants.
 - Mean AUC across regimes ≈ 0.60 vs. ≈ 0.58 for the plain CNN.
- **Vanilla CNN peaks in *Extended*.**
 - Achieves AUC = 0.66 (highest single AUC reported) and ACC = 0.56, showing that additional context/sensor channels help separability.
- **Weighted ResNet shows modest gains (only one worthy to be shown).**
 - On *Standard 1*: acc = 0.57, AUC = 0.61— better than the CNN’s 0.52/0.58 in the same regime.
 - Overall easy overfitting on the train dataset can lead to the assumption that the increasing complexity of the model wouldn’t give better results
- **Persistence Case is essentially random.**
 - All models collapse to ~ 0.48 ACC and ~ 0.48 AUC when only long-term, low-bandwidth signals are available.

2.2.3 Chosen Models

Overall, it was considered to pick **w-CNN Extended 3** (“Vivid Nebula”) due to the good labels balance and acceptable binary separation (see Fig. 2.3 and Fig. 2.4). But due to time constraints reasons, and also good class balance – **CNN Extended 1** (“Silent Raven”) was chosen for further experiments with an ensemble classifier (see Fig. 2.5 and Fig. 2.6).

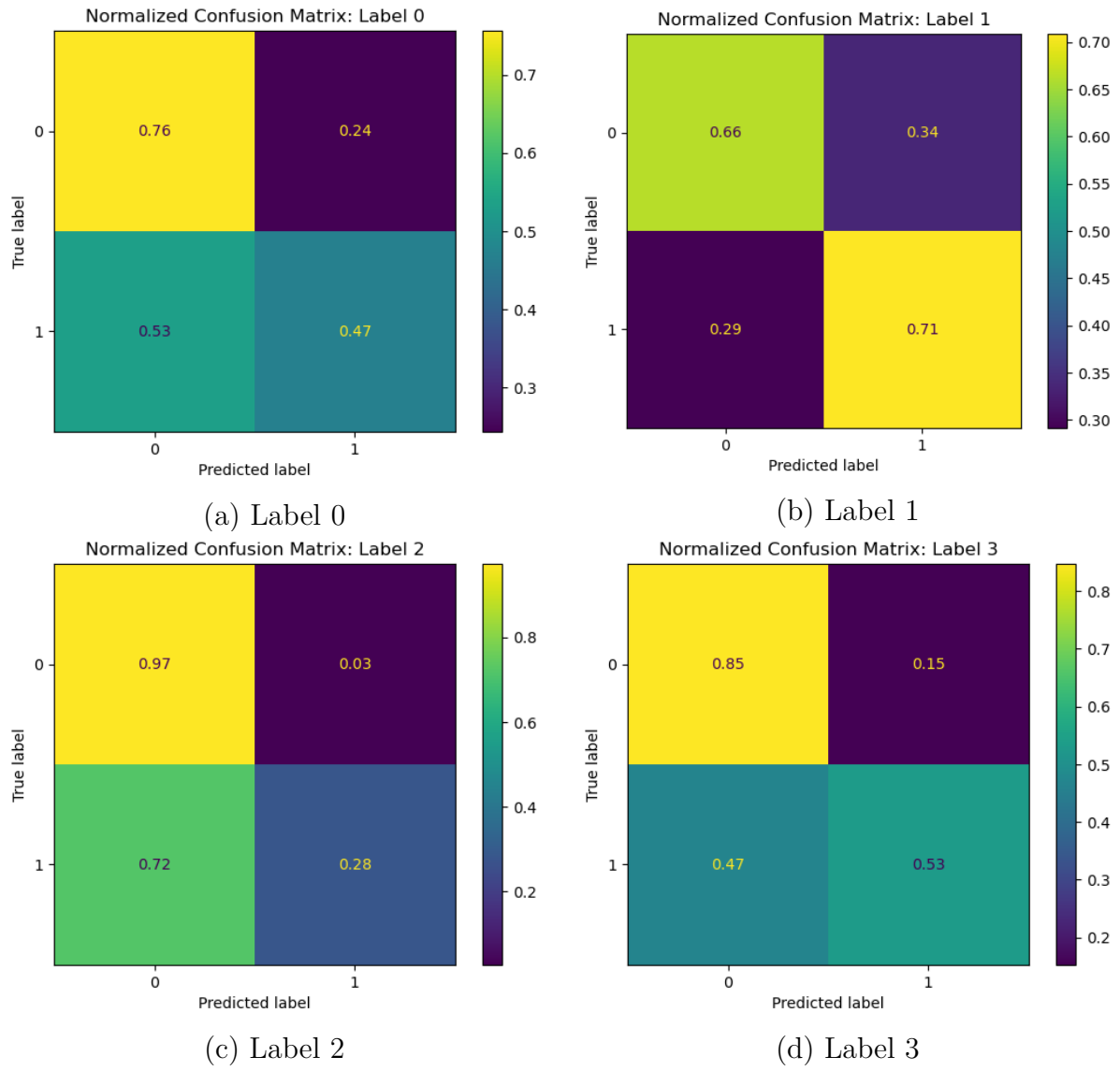


Figure 2.3: w-CNN Extended 3 ("Vivid Nebula"), per label Confusion Matrix.

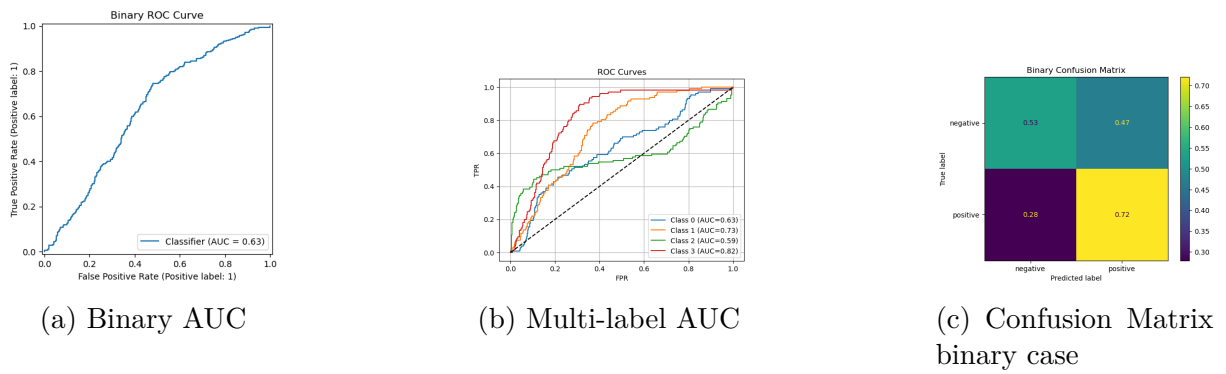


Figure 2.4: w-CNN Extended 3 ("Vivid Nebula"), other

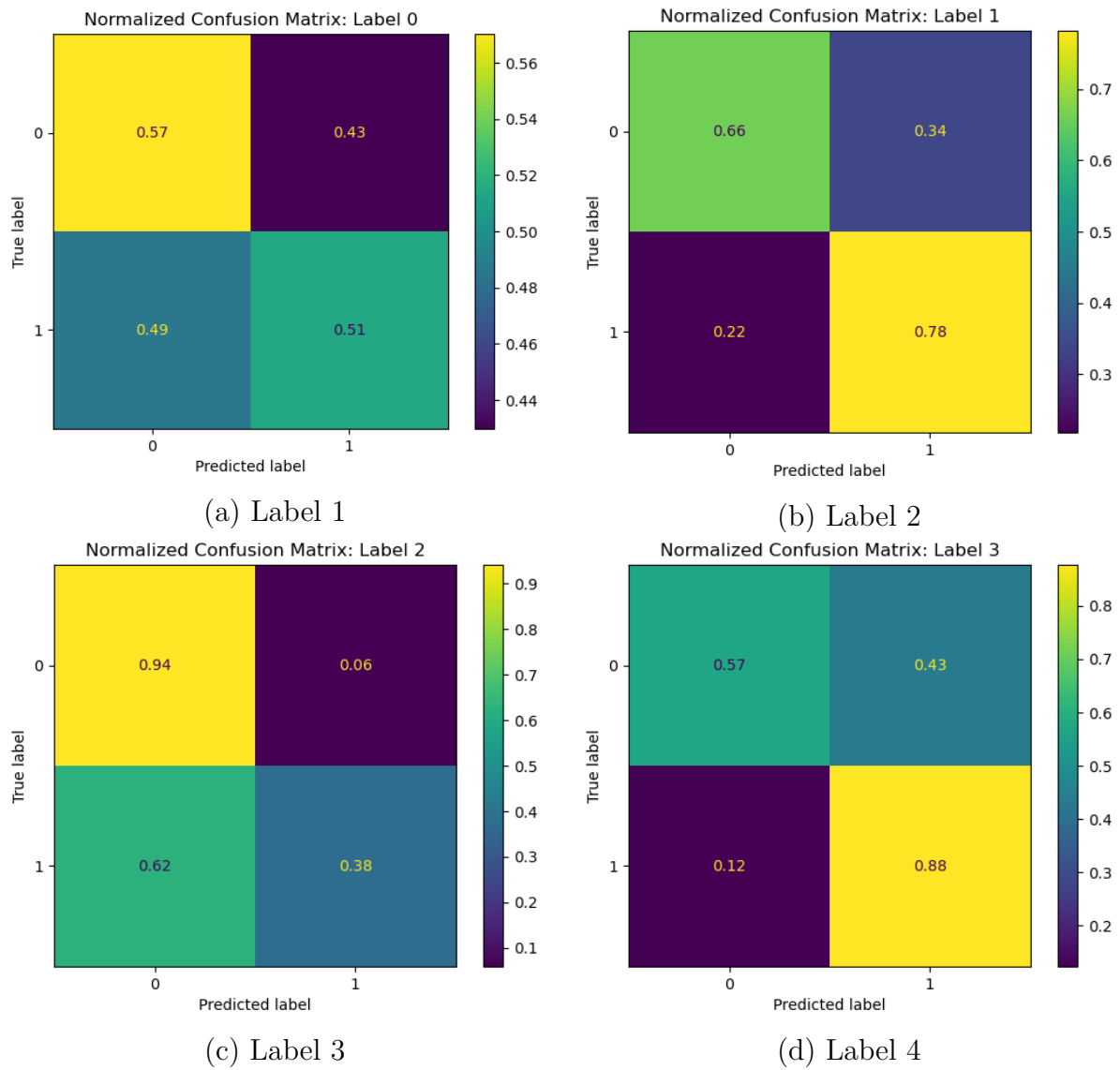


Figure 2.5: CNN Extended ("Silent Raven"), per label Confusion Matrix.

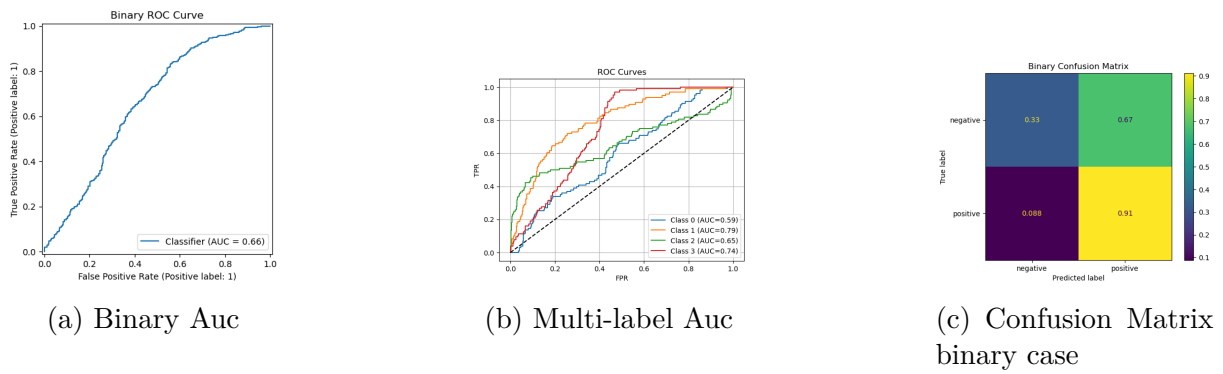


Figure 2.6: CNN Extended ("Silent Raven"), other

2.3 LSTM

An LSTM network is a type of Recurrent Neural Network (RNN) particularly suited to capturing dependencies in sequential data by retaining relevant information across longer periods. Unlike conventional RNNs, LSTMs effectively address the issues of vanishing or exploding gradients through the use of specialized gating mechanisms. These gates—the forget gate (f_t), input gate (i_t), and output gate (o_t)—dynamically manage the internal cell state (C_t), selectively deciding what information to keep, discard, or update as the sequence progresses. As shown in Figure 2.7, the intricate interactions between these gates and their activation functions (sigmoid σ and tanh) allow the LSTM units to adaptively learn and retain essential temporal patterns. Consequently, LSTM networks are highly effective in modeling complex time-dependent data, such as sensor signals involved in vehicle braking analysis.

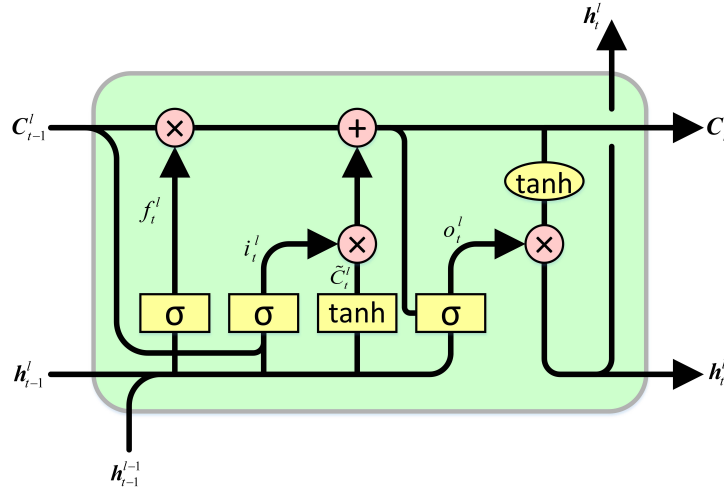


Figure 2.7: LSTM Architecture : Detailed schematic of a standard LSTM unit, illustrating the interactions between gates, cell state, hidden state, and activation functions (sigmoid σ , tanh) [5]

LSTM networks are widely used for time-series classification tasks due to their ability to model temporal patterns in sequential data. They have been applied in domains such as activity recognition [8], fault detection [6], and vehicle behavior modeling [10]. This study focuses on classifying braking conditions trained on sensor time series and supporting tabular features, such as vehicle speed, weight, and average deceleration. The classification task have twelve brake states, each corresponding to a different combination of conditions of four brake positions: front-left, front-right, rear-left, and rear-right. In selected experiments, this multi-class problem is reformulated as a multi-label classification task for finer resolution.

2.3.1 Methodology

In addition to the model architecture, several aspects of the training pipeline were explored, including input normalization, under sampling to address class imbalance, class weighting in the loss function, and basic data augmentation. Hyperparameter tuning was performed using both random and Bayesian search strategies. The combination of sequential and tabular data provided a way to examine how different input features relate to observed braking conditions.

Across all experiments, the input data was normalized—sensor sequences were scaled using a robust scaler and clipped, while tabular features were standardized using z-score normalization.

The original dataset shows a clear class imbalance, as seen in Figure 2.8. Most of the samples belong to class 0, which represents fully functioning brakes, while the remaining classes are underrepresented. This imbalance can lead the model to favor the dominant class during training, reducing its ability to detect faulty braking conditions. To address

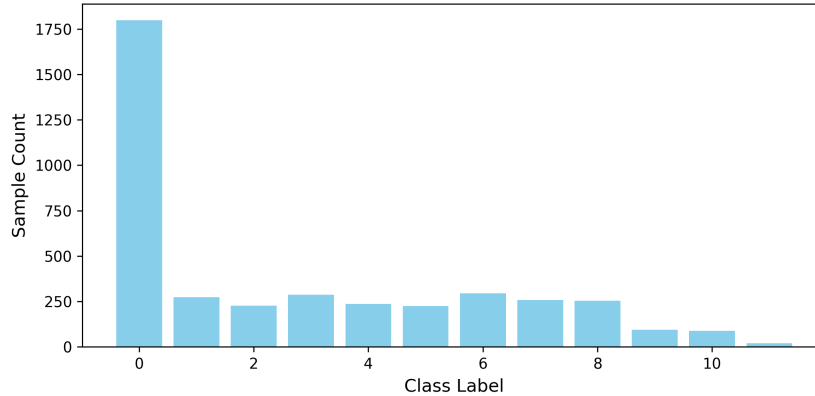


Figure 2.8: class distribution of the Training data

this, different techniques were applied. Data augmentation was used to increase the number of samples in the minority classes. This was done in three stages—first increasing each minority class to 500 samples, then to 1250, and finally to 1600—while keeping class 0 unchanged. In another experiment, undersampling was used to reduce the number of class 0 samples to 500, keeping all other classes the same. Additionally, a weighted random sampler was tested to give more importance to rare classes during training without changing the data distribution. These methods helped balance the training process and gave the model a better chance to learn from underrepresented brake conditions.

After balancing the data, the model was trained using a hybrid LSTM-based architecture. The LSTM layer processes the sequential sensor data to learn patterns over time, while the tabular inputs are passed through a small feedforward layer with ReLU activation and dropout. The outputs from both parts are then combined and passed through a classification head to predict the braking condition.

To improve performance, several model parameters were tuned, such as the number of LSTM layers, hidden size, dropout rate, and whether the LSTM is bidirectional. A combination of random search and Bayesian optimization was used to explore these settings. The model was trained using the Adam optimizer and cross-entropy loss. For evaluation, both random train-test splits and k-fold cross-validation were used across the different case studies to ensure robustness and fairness in performance comparison. In the single-label classification experiments, performance was measured using accuracy and F1 score. Depending on the case study, the weighted or macro F1 score was reported. In the multi-label setting, each of the four brake positions—front left, front right, rear left, and rear right—was treated as a separate binary classification problem. F1 scores were computed individually for each brake to assess how well the model captured irregularities at specific positions. To study how different training settings affect model performance, eight case studies were carried out using the LSTM model. These cases vary in hyperparameter tuning method, data augmentation, class distribution, and whether the task was

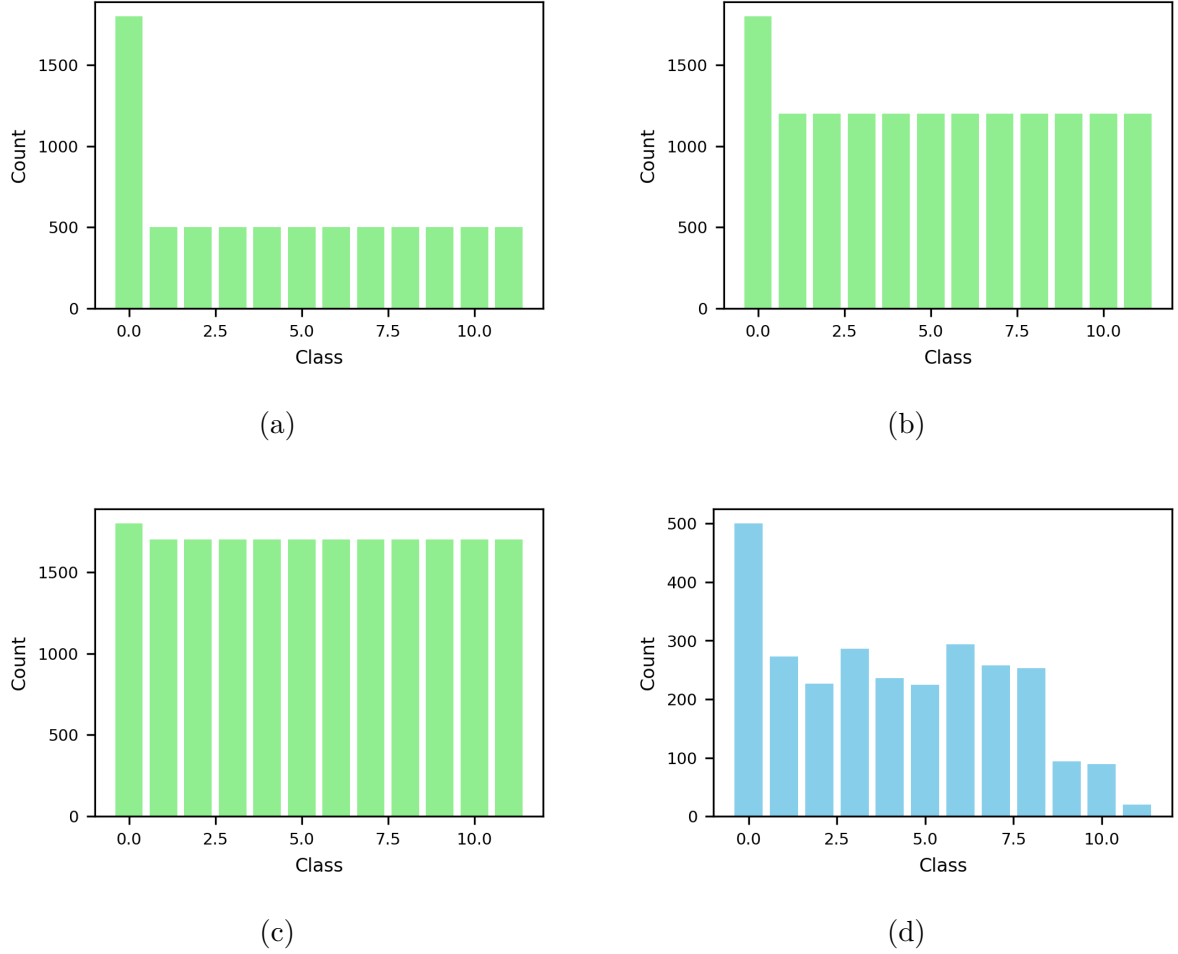


Figure 2.9: Class distribution after different balancing strategies. (a) Augmentation to 500 samples , (b) augmentation to 1250 samples, (c) augmentation to 1600 samples per minority classes, and (d) undersampling of class 0 to 500 samples.

single-label or multi-label. Figure 2.10 shows an overview of all case studies, each with a different combination of settings used during training and evaluation.

2.3.2 Results and Analysis

The eight LSTM case studies were designed to test how different training setups affected the model’s ability to classify braking conditions.

Normalization helped to keep the training stable, especially when handling very different input types. Two kinds of hyperparameter tuning were used: random search and Bayesian optimization with Optuna. In Case 2, Optuna was combined with class weights in the loss function to handle class imbalance. But, this setup still led to the lowest F1 score among all cases (0.079), showing that tuning alone wasn’t enough to overcome the imbalance, especially without any form of data balancing like augmentation or resampling.

Random search was used in most other cases and worked well when combined with data augmentation. In Cases 3 to 5, minority classes were augmented in different proportions. Case 4, where the smaller classes were increased to 1250 samples each, gave the best F1 score overall (0.4755). A moderate level of augmentation helped the model generalize better, while too little or too much either didn’t solve the imbalance (Case 3) or possibly

Case No.	Model	Hyper-parameter tuning	Data Augmentation	Data Distribution	Multi label Classification	F1 score	Accuracy
1	LSTM	Random	No	As it is	No	0.3729	0.33
2	LSTM	Bayesian Optimization	No	As it is	No	0.079	0.3716
3	LSTM	Random	Yes	Label 0 = 1750 Other labels = 500	No	0.4500	0.4096
4	LSTM	Random	Yes	Label 0 = 1750 Other labels = 1250	No	0.4755	0.3766
5	LSTM	Random	Yes	Label 0 = 1750 Other labels = 1600	No	0.4565	0.3986
6	LSTM	Random	Under sampled	Label 0 = 500	No	0.3259	0.2867
7	LSTM	Random	No	As it is	Yes	front left : 0.03 , front right : 0.21 , rear left : 0.20 , rear right : 0.33	front left : 0.80 , front right: 0.87 , rear left : 0.79 , rear right : 0.83
8	LSTM	Random	No	As it is	Yes	front left : 0.02 , front right : 0.13 , rear left : 0.23 , rear right : 0.26	front left : 0.77 , front right : 0.82 , rear left : 0.81 , rear right : 0.64

Figure 2.10: Results of LSTM

introduced noise or redundancy (Case 5).

The confusion matrix for Case 4 (Figure 2.11) shows that while the model correctly classifies many samples from class 0, a notable portion of minority class samples—particularly class 4 and 5—are also misclassified as class 0. This indicates a lingering bias toward the dominant class, even with augmentation. Correct predictions for minority classes like 1 and 2 are visible, but the overlap between some faulty and normal brake signals remains a challenge. Undersampling was tried in Case 6 by reducing the size of the dominant class to match the smaller ones. While this made the class distribution more even, the model had less data to learn from overall. As a result, both the accuracy and F1 score were lower compared to the augmented cases.

Regarding evaluation, random-split validation was used during training. Full k-fold cross-validation was avoided, especially in cases with augmented datasets, because of how much time and memory it would take to train those models repeatedly. The random-split method provided reliable estimates while being manageable in terms of compute.

It is also important to note that the test set only contains labels from 0 to 5. Class 0 dominates the distribution, while the remaining labels are equally underrepresented. This imbalance in the test set can influence evaluation results, as the model is never tested on the full label space seen during training.

Cases 7 and 8 tested a multi-label version of the problem, where each brake position was predicted separately. Accuracy was generally high for all four brakes (above 0.75), but the F1 scores were lower—especially for the front brakes. This shows that while the

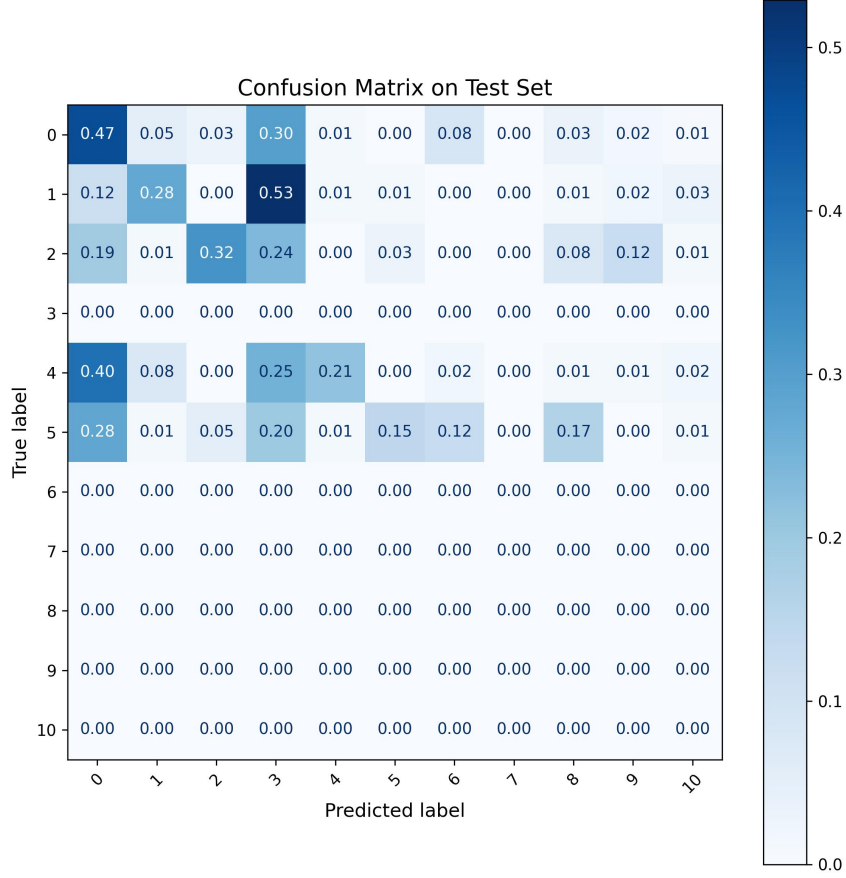


Figure 2.11: **Normalized confusion matrix for Case 4**, evaluated on the test set using the best LSTM model. Each cell represents the proportion of true samples from a given class predicted as each possible label

model was often correct overall, it still had trouble detecting more subtle or rare faults at specific positions.

Overall, the results suggest that balancing the dataset had the greatest impact on improving model performance. Among the tested approaches, random hyperparameter search combined with data augmentation produced the most consistent and reliable outcomes. In contrast, more advanced methods like Bayesian optimization and multi-label classification introduced additional complexity but did not consistently lead to better results—particularly when data imbalance or limited sample diversity remained an issue.

2.4 Additional Models

2.4.1 Transformer Models

Transformer architectures, initially developed for natural language processing, have been adapted for time-series applications due to their ability to model temporal dependencies using self-attention mechanisms. In this study, in addition to the LSTM-based experiments, two case studies were conducted using a Transformer-based architecture. The model architecture consisted of a Transformer encoder to process time-series sensor inputs, combined with a feedforward layer for the tabular features. These were joined and

Case No.	Model	Hyper-parameter tuning	Data Augmentation	Data Distribution	Multi label Classification	F1 score	Accuracy
9	Transformer	Random	Yes	Label 0 = 1750 Other labels = 500	No	0.4240	0.3526
10	Transformer	Random	Yes	Label 0 = 1750 Other labels = 500	Yes	front left : 0.30 , front right : 0.11 , rear left : 0.13 , rear right : 0.15	front left : 0.81 , front right : 0.82 , rear left : 0.87 , rear right : 0.76

Figure 2.12: Transformer models Case Studies and Results

passed through a classification head. Random hyperparameter tuning was applied, and both cases used augmented data with 1750 samples for class 0 and 500 samples for all other classes.

Case 9 followed a single-label classification setup, similar to earlier LSTM cases, while Case 10 used a multi-label formulation, predicting the state of each brake position independently. This allowed for direct comparison of classification strategies under consistent data conditions 2.12.

Case 10 aimed to assess whether predicting each brake position individually could offer better fault localization. The evaluation results are presented in Figure 2.13b, showing normalized confusion matrices for each of the four brake labels. Accuracy scores across the four positions ranged from 0.76 to 0.87, while F1 scores remained lower, especially for front-right and rear-left. While the model could identify the normal brake conditions with high confidence, it still struggled to detect less frequent faults consistently. Together, these experiments show that while Transformer-based models can capture meaningful temporal and contextual patterns, their performance is still limited by data imbalance and overlap among fault categories. The multi-label setup offered additional interpretability by decoupling predictions across brake positions, but detection of rare events remained a challenge even with data augmentation.

2.4.2 XGBoost with Classifier Chains

As part of the additional experiments, a classical machine learning approach was implemented using XGBoost with classifier chains is implemented. Unlike the LSTM and Transformer models, this method does not rely on temporal dynamics, but instead focuses on structured feature engineering.

The data preparation involved imputing missing values using k-nearest neighbors, scaling of both tabular and sensor data. From the sensor signals, statistical features were extracted, along with frequency-domain features obtained by fast Fourier transform. These were combined with the tabular features to form the final input set.

Labels were converted into a binary multi-label format, with one output for each brake position. To address class imbalance, oversampling was applied .

The classification stage used an ensemble of classifier chains, each built with XGBoost as the base model. Each chain predicted the labels in a specific order, passing earlier predictions as additional inputs to the next. Different random label orders were used to create diversity across chains, and the final outputs were obtained by averaging their

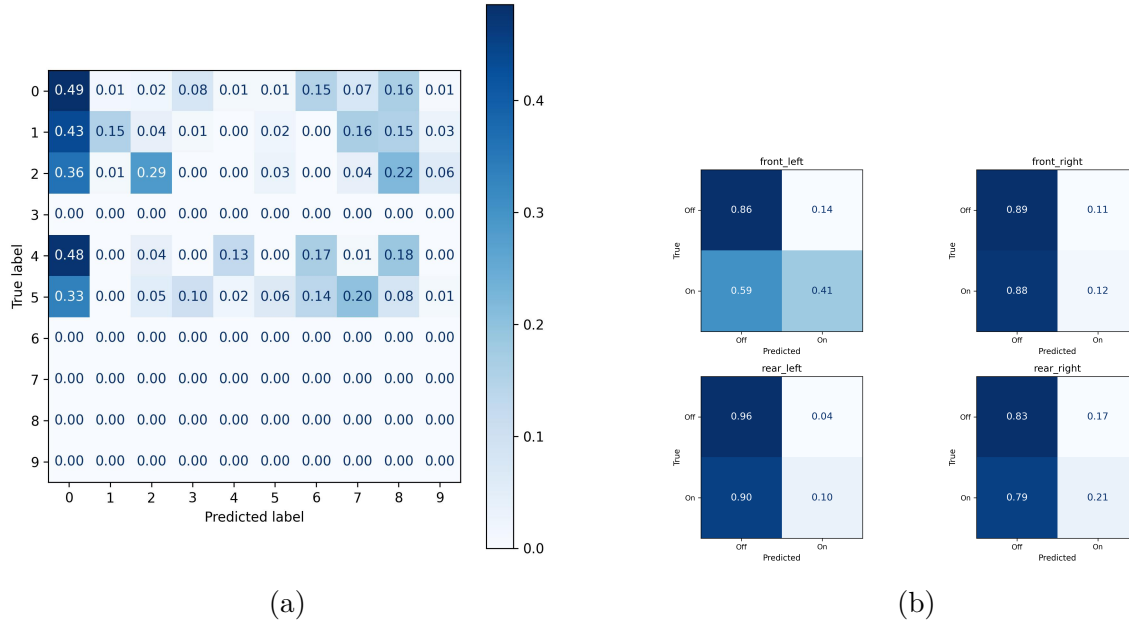


Figure 2.13: (a) Normalized confusion matrix for Transformer single-label classification. (b) Normalized confusion matrices for each brake position in multi-label classification. Together, these illustrate classification behavior across different problem setups.

predictions and applying label-specific thresholds. This approach aimed to explore how far structured features and label dependency modeling can support multi-label classification without deep temporal models.

Case No.	Model	F1 score	Accuracy
11	XGB with classifier chains	front left : 0.25 , front right : 0.33 , rear left : 0.36 , rear right : 0.23	front left : 0.62 , front right : 0.72 , rear left : 0.79 , rear right : 0.67

Figure 2.14: Per-label F1 score and accuracy of the XGBoost with classifier chains model (Case 11) for each brake position.

The XGBoost with classifier chains model (Case 11) achieved reasonable performance given the complexity and imbalance of the dataset. Label-wise accuracies range from 0.60 to 0.80, and F1 scores for each label range from 0.21 to 0.36 (see Figure 2.14). Considering the multi-label case studies (7, 8, and 10), XGBoost yields the highest F1 score across all four brake positions.

Interestingly, the confusion matrices (Figure 2.15) show that the model can distinguish between faulty and non-faulty conditions with over 60% accuracy in nearly all brake components, except for the rear right faulty case. These results suggest that even a non-deep learning method like XGBoost, when combined with structured feature engineering and ensemble strategies, can offer a competitive baseline in multi-label fault classification under challenging conditions.

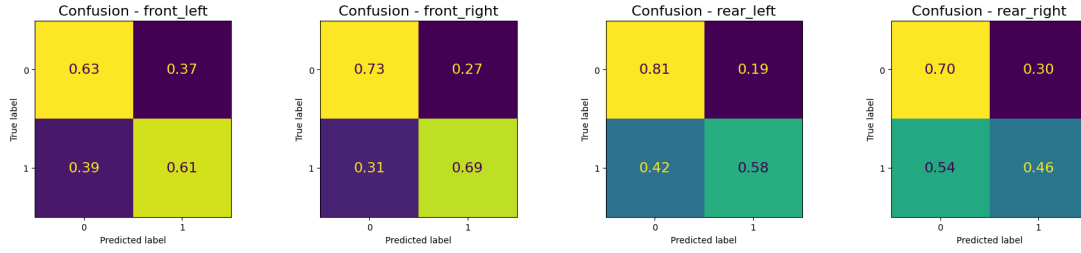


Figure 2.15: Normalized confusion matrices for each brake position predicted by the XGBoost classifier chain model. The values represent class-wise prediction distributions across normal and faulty labels.

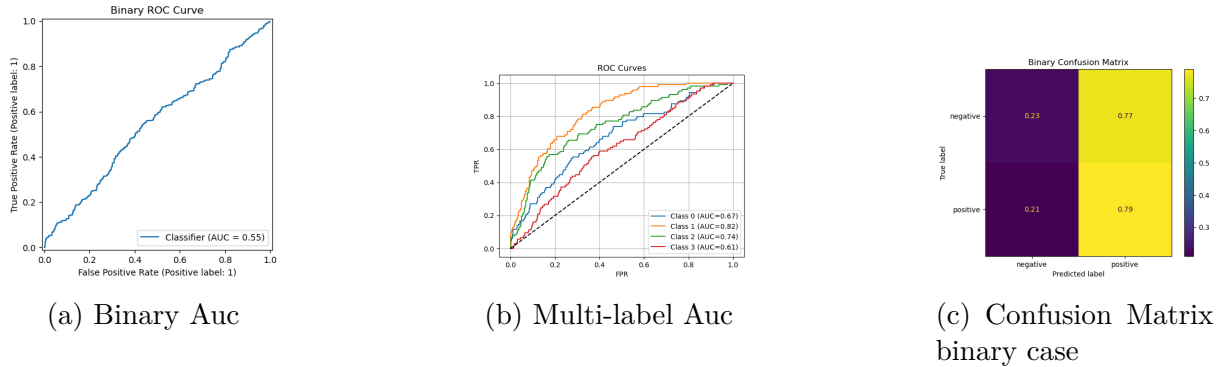


Figure 2.16: XGBoost, other

2.5 Best Model

Final model consideration was between **Anupama's** and **Oleksandr's** models, so concidiration was given to **Sara**.

After testing different neural network architectures and classical models, we selected the **w-CNN Extended 3** (“**Vivid Nebula**”) as the best model for brake system classification. This choice is based on both performance and the main goal of the project, which is to use neural networks for fault detection.

The “Vivid Nebula” model is a weighted convolutional neural network (CNN) trained on an extended dataset with extra sensor channels and features. According to the results table (Table 2.2), this model achieves a binary accuracy of 0.61 and a macro-F1 score of 0.32. These scores are important because the dataset is imbalanced, and macro-F1 shows how well the model treats all classes, not just the most common one.

The model also shows good class separation in both binary and multi-label classification, as seen in the confusion matrices and ROC curves (Figure 2.3 and Figure 2.4). The binary AUC reaches up to 0.66, and the model is especially good at detecting faulty brakes. This clear separation is important for safety-critical systems like braking.

Other models were also tested. The LSTM with moderate data augmentation (Case 4) achieved a slightly higher F1 score in some tests, but the “Vivid Nebula” CNN was more consistent and robust, especially in the binary case. Transformer models were also tried, but they did not outperform the CNN or LSTM. FCN (Fully Convolutional Networks) were considered as well, but their results were not as good as the weighted CNN, especially for binary class separation and robustness.

The **XGBoost** with classifier chains model gave competitive results, especially for the F1 score per brake position. However, the model has a bigger bias towards the positive

class comparative to CNN (see Fig. 2.16), also the primary goal of the project is to use neural networks, we chose the CNN model.

In summary, the **w-CNN Extended 3 (“Vivid Nebula”)** was selected because it gives the best balance of class separation, binary accuracy (0.61), macro-F1 (0.32), and fits the project’s focus on neural networks. While other models, such as XGBoost and LSTM, also performed well, the weighted CNN is the most suitable choice for this task.

Chapter 3

Generative AI

In this chapter, we examine whether the use of generative AI for dataset expansion improves the classifiers in the previous chapters. For this, we will implement a Conditional Generative Adversarial Network (CGAN) and a Recurrent Neural Network (RNN).

3.1 Visualization

Evaluating and quantifying the performance of generative models is a major challenge. Straightforward statistical performance metrics like those for classifiers and regression models do not exist because there is no objective loss function. Some commonly used metrics include Fréchet Inception Distance (FID) and Kernel Inception Distance (KID), but they are not straightforward to implement and require training another model to compute. Therefore, I will rely on manual assessment to determine the quality of models. Likewise, in the upcoming section, I will extend the dataset with synthetic data. This may also serve as a benchmark, i.e. generated data better reflects the real underlying distribution if the classifier trained on it achieves better results on the test dataset. Static graphs of the accelerometer data don't make it easy to visually assess the generation quality, therefore I developed a program to visualize samples from the dataset by creating a top-down 2-dimensional animation of the car's trajectory and rotation using Grant Sanderson's Manim module.

3.1.1 Mathematical Foundation

For the animation, we need the car's position and rotation at every point in time. There are two problems to solve:

1. Convert acceleration vectors to position vectors.
2. Convert rotational velocity vectors to absolute rotation vectors.

Acceleration to Position

Verlet integration lends itself well to this. Let $x(t)$ be the position at time t , and assume it is twice differentiable, then performing a Taylor expansion forwards and backwards, we get:

$$x(t + \Delta t) = x(t) + \dot{x}(t)\Delta t + \frac{1}{2}\ddot{x}(t)\Delta t^2$$

and also

$$x(t - \Delta t) = x(t) - \dot{x}(t)\Delta t + \frac{1}{2}\ddot{x}(t)\Delta t^2$$

Then

$$x(t + \Delta t) + x(t - \Delta t) = 2x(t) + \ddot{x}(t)\Delta t^2 \iff x(t + \Delta t) = \ddot{x}(t)\Delta t^2 - x(t - \Delta t) + 2x(t)$$

Converting this to the discrete case, we get the recurrence $x_{n+1} = a_n\Delta t^2 - x_{n-1} + 2x_n$, where:

- x_n is the position at time n
- Δt is the timestep
- a_n is the acceleration at time n from the dataset

Hereby, we define the initial position $x_0 := (0, 0, 0)$. In order to be able to evaluate the recurrence we also lack the x_1 , i.e. we lack information about the initial velocity. If the braking measurements always stop at velocity 0, we can retrieve the start velocity by performing this process in reverse. But this can not be guaranteed, therefore, I will instead take the possibly inaccurate starting velocity from the metadata (scalar) v_{start} and set $x_1 = x_0 + (v_{\text{start}}, 0, 0)\Delta t$, setting the coordinate system such that the vehicle initially moves in the positive x direction.

Rotational Velocity to Rotation

Here, we have given the first derivative, therefore Verlet integration is not appropriate. Let $r(t)$ be the rotation at time t . We have

$$r(t + \Delta t) = r(t) + \dot{r}(t)\Delta t$$

Or in the discrete case:

$$r_{n+1} = r_n + v_n * \Delta t$$

r_0 is unknown. The car will be assumed to be rotated towards its initial travelling velocity (towards positive x).

3.1.2 Results

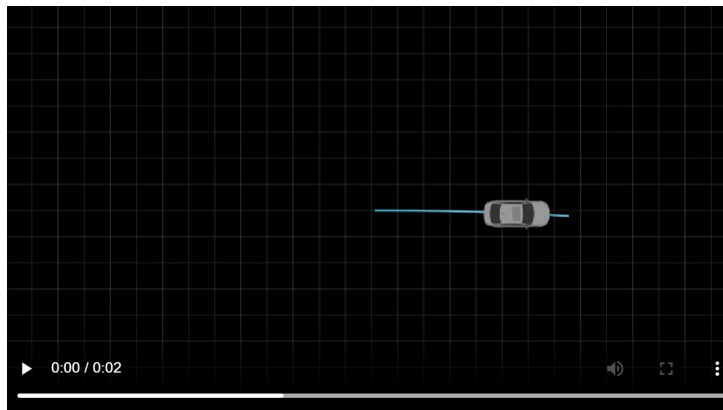


Figure 3.1: Single frame from a generated video of a sample generated by the CNN GAN.

3.2 GAN

3.2.1 Theory

In the traditional GAN architecture, it is not possible to meaningfully guide generation based on labels. The generator's only input is a latent seed vector which is then mapped to a sample intended to resemble data from the target distribution. It is however not known which regions of the latent space result in which type of sample. Therefore I've chosen to implement a Conditional GAN to condition generation on labels [7].

The generator is a function $G_\theta(y, z) \mapsto (x_0, x_1, x_2, \dots, x_n)$ where z is an m -dimensional vector in $\mathbb{R}^m$ representing the latent vector and y is a label encoding the brake manipulation data. It is trained by minimizing $\log(1 - D_\phi(G_\theta(z)))$.

The discriminator is a binary classifier $D_\phi(y, x_i) \mapsto \mathbb{P}(\text{real}|y, x_i)$ trained by maximizing the log-probability:

$$\mathbb{E}_x[\log D(x|y)] + \mathbb{E}_z[\log(1 - D(G(z|y)))]$$

3.2.2 Methodology

Training was initially quite unstable because the discriminator would learn vastly faster than the generator. This is a common problem when training GANs and leads to vanishing gradients and a failure of convergence [3].

To remedy this, we apply label smoothing to prevent full convergence of the discriminator. Practically, this means that during training we assign a probability lower than 1 to the real data in the loss function. This made training more stable, even if the discriminator initially dominated. I've tried two different model architecture: a classic MLP for both generator and the discriminator and a Convolutional Network for the discriminator and a Deconvolutional Network for the generator.

3.2.3 Results

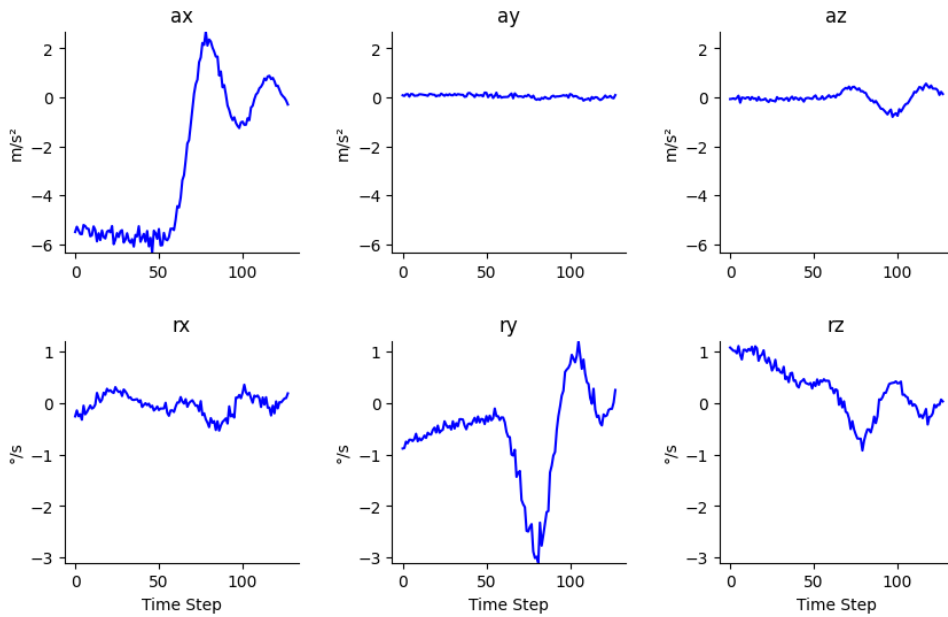


Figure 3.2: Example generation of the MLP GAN (Conditioned on label 5)

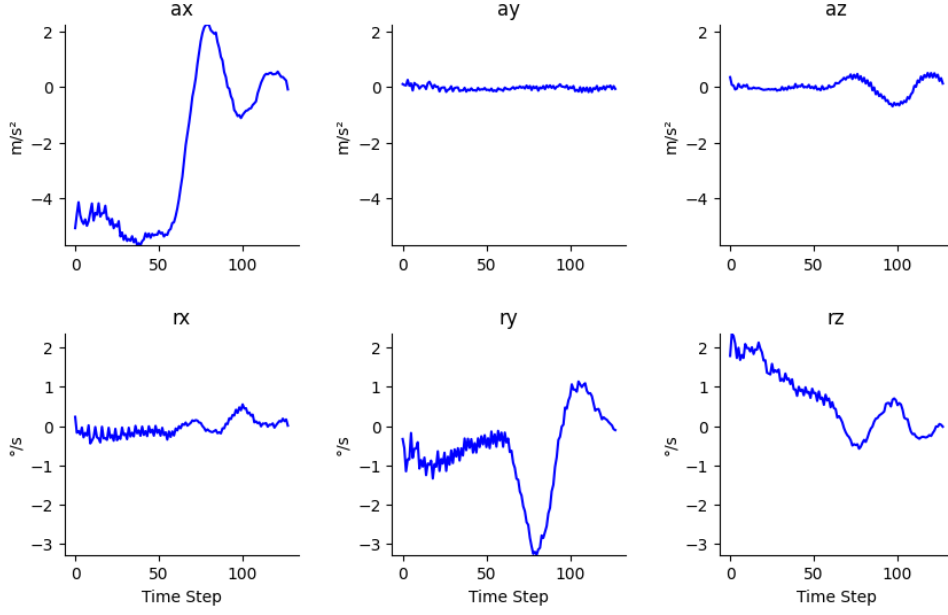


Figure 3.3: Example generation of the CNN GAN (Conditioned on label 5)

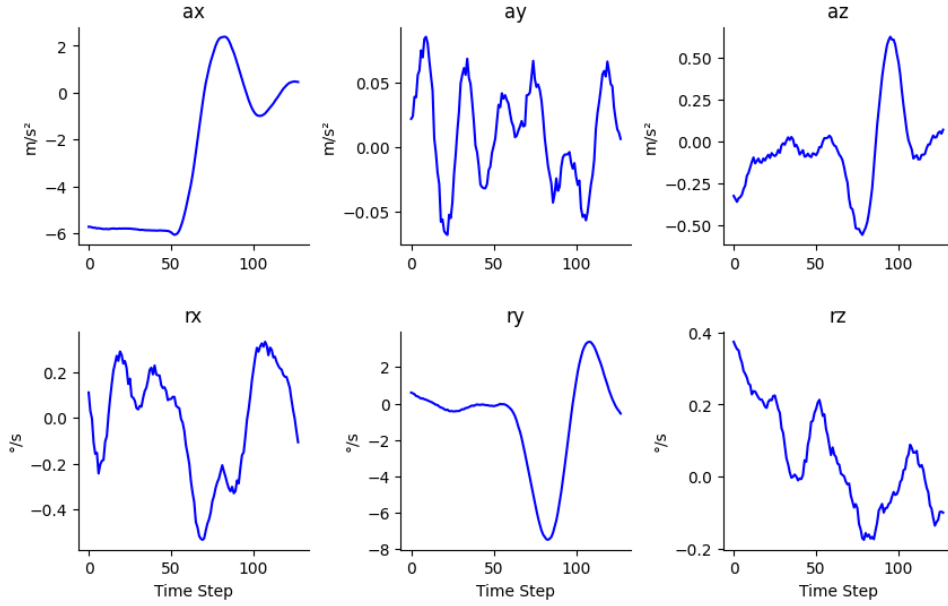


Figure 3.4: Real Data (Example from label 5)

Upon visual inspection, the generations look very similar. The CNN GAN has higher-amplitude, more periodic looking noise which is likely a result of the underlying architecture. Unlike the CNN GAN, the MLP GAN has no smooth regions and lower-amplitude noise. Based on this, we were not able to determine which architecture produces better results.

3.3 RNN

3.3.1 Methodology

The RNN I implemented is a function $f_\theta(y, (x_0, x_1, \dots, x_n)) \mapsto (x_1, x_2, \dots, x_{n+1})$ mapping a label y and previous sequence to a new sequence which drops the first value and predicts a new last value. Training is done by minimizing $\frac{1}{n}e^\top e$ where $e = (x_1, x_2, \dots, x_{n+1}) - f_\theta(y, (x_0, x_1, \dots, x_n))$. I've chosen to use an LSTM over a traditional RNN as it lead to slightly better results.

3.3.2 Results

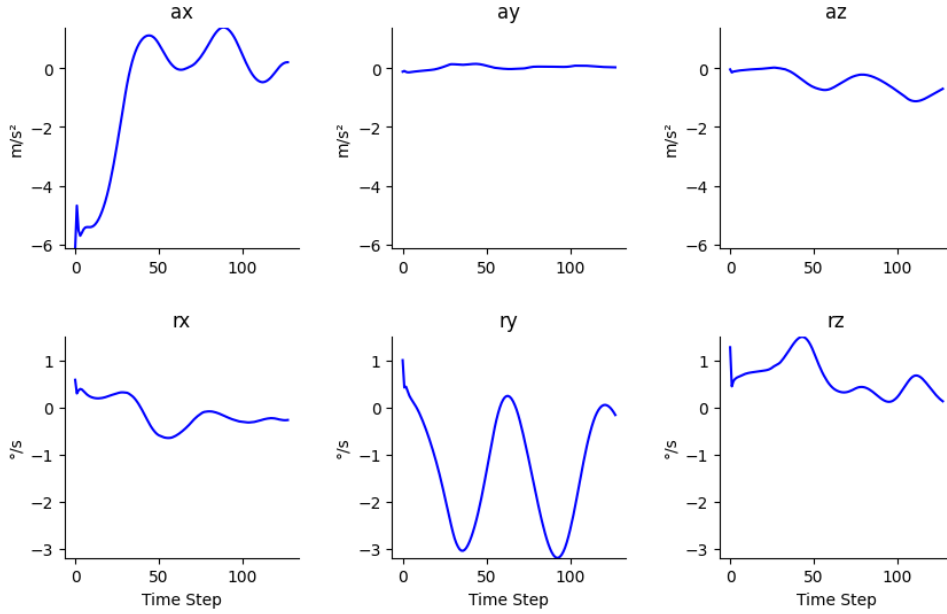


Figure 3.5: Example generation of the RNN (Conditioned on label 5)

The generated data looks very smooth and at times seems to overshoot slightly. Overall, the values are still in believable ranges and the higher level changes still appear close to those found in the original data. Additionally, all axis show a jump in the beginning of the sequence. This could be due to a lack of context and does not appear in the training data.

3.4 Comparison

Both the GANs and the RNN are suitable for generating artificial data that resembles the original distribution, although GANs were better at capturing the finer details and overall appear more realistic while the RNN's generations appear overly smooth. Therefore the generations of the GANs seem much better than those of the RNN. Using the RNN to predict a normal distribution instead of directly predicting the next element in the sequence would likely help with the problem of overly smooth generations. This distribution can then be sampled during generation and naturally result in noisier outputs as found in the training data.

3.5 Extending the dataset

3.5.1 Methodology

As the GAN's generations are more promising, I've chosen to use the CNN GAN to extend the dataset. The first step for this is to package the generated sensor data up to be compatible with the rest of the code base. For this, I simply take the median of the deceleration average, mass and velocity and choose the fitting name 'Fake Car 9000' for the model column.

The generated labels are uniformly distributed. This leads to a more balanced data set as a whole.

For more comparable results, I will not perform any form of hyperparameter tuning and simply attempt retraining the base LSTM with a balanced model architecture.

3.5.2 Results

Performance

Before data set extension, the model yields an F1 score of 0.3610 with the following confusion matrix:

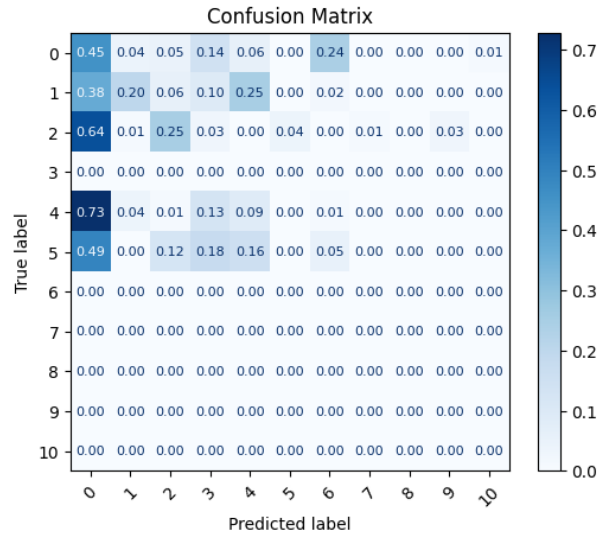


Figure 3.6: Confusion Matrix of the base LSTM with the default dataset

After training with a dataset twice the size of the original consisting of 50% synthetic data, the model yields an F1 score of 0.4306 with the following confusion matrix:

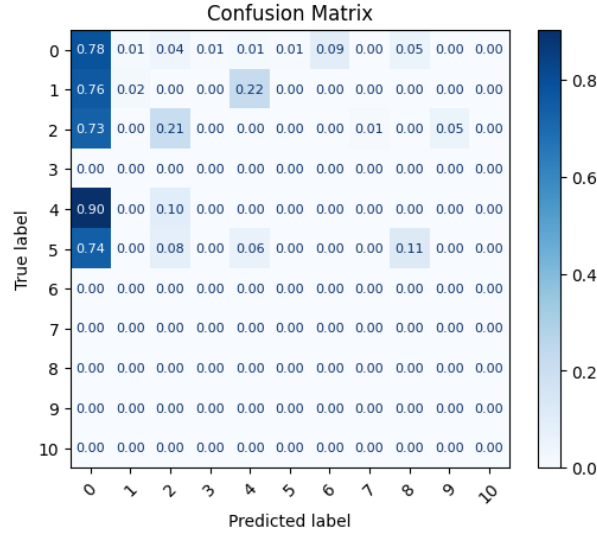


Figure 3.7: Confusion Matrix of the base LSTM with the 50-50 dataset

At last, I trained a model on a large, fully-synthetic dataset of 25000 entries. This only gave an F1 score of 0.0736 and lead to the following confusion matrix:

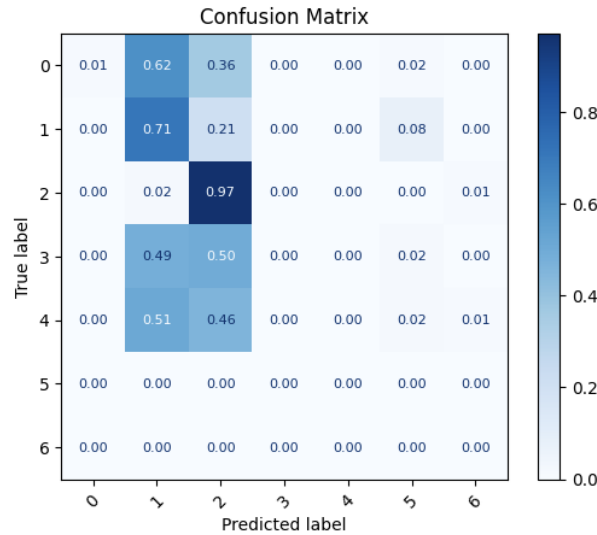


Figure 3.8: Confusion Matrix of the base LSTM with the large fully-synthetic dataset

Discussion

It can be seen that the 50-50 dataset leads to a better F1 score than the original dataset, while the fully synthetic one collapses, predicting 2 almost every time. This is likely an indication that the CGAN has not learned the conditions well.

GANs have been used successfully for data augmentation [4], however there seems to be some theoretical justification for why synthetic data generated by generative models can harm performance [9].

From a theoretical standpoint, both the GAN and classifier model have the same data to work with and both cannot learn information not represented in the dataset. The GAN will only generate data based on the information already found in the dataset and therefore not add any new information.

But from an empirical standpoint, GANs can still help training by providing high-quality interpolation between the existing data points, i.e. by densifying the dataset. They also introduce some level of noise, which in itself is a common data augmentation technique and can increase the robustness of the model.

Chapter 4

Ensemble Models

4.1 Motivation

By the assumption that **XGBoost Chains** and **CNN** use different “patterns” in data due to their architectural difference, it was decided to experiment with an ensemble model of two of those. Given that models were performing better on different labels over each other; this also provides practical motivation for the ensemble.

4.2 Candidate Ensemble Techniques

Table 4.1: Summary of ensemble candidates.

ID	Technique	Core Idea
E1	Calibrated Mean	Arithmetic mean of the two probability vectors after temperature scaling each base model.
E2	Label-Weighted Mean	Learn fixed weights $w_{\text{RF}}, w_{\text{CNN}}$ per label on the validation split and blend $p = w p_{\text{XGB}} + (1 - w) p_{\text{CNN}}$.
E3	Gated Selector	Linear regression outputs a gate $g \in [0, 1]$ per label and sample; final $p = g p_{\text{XGB}} + (1 - g) p_{\text{CNN}}$
E4	Logit Stacking	Concatenate both probability vectors and train a logistic-regression meta-classifier.
E5	Dempster-Shafer Fusion	Interpret probabilities as mass functions, apply DS rules to combine evidence while discounting conflict.
E6	Rank Averaging	Convert probabilities to ranks per label, average ranks, then rescale to $[0, 1]$.

4.3 Results

1. Clear overall winner – Gated Selector

Achieves the best *Binary-ACC* (0.58) and the top *micro-F1* (0.33). Its per-label scores are balanced—especially on the difficult labels L1 and L2, both at 0.40—showing that the adaptive gate reliably selects the stronger base model *per instance*. Improvement over all other models is well-balanced binary confusion matrix (see Fig. 4.2c).

2. Tight second tier – Label-Weighted Mean & Stacking

Both cluster around 0.63 *Binary-F1*. They lag behind the gate on the two fist labels and *ACC*.

3. Evidence fusion surprise – Dempster–Shaffer (DS)

Although DS slips to 0.55 F1, it posts the **second-best micro-F1** (0.31) and the *highest* per-label F1 on L2 (0.47), implying that its conflict-discounting mechanism is advantageous when one base model dominates the other with high confidence.

4. Rank Averaging under performs but maximizes “samples” F1

Dropping probability scale hurts rare-class precision, driving micro-F1 down to 0.26; however, it *leads* on the samples-level metric (0.20).

5. Per-label insights

- L2 is the easiest class across all methods (0.35–0.47 F1).
- L0 and L3 remain challenging—no method exceeds 0.32 on L3 or 0.31 on L0.
- The gate lifts L0 by almost 0.10 F1 over simple weighting, confirming its intended behavior.

Table 4.2: Test-set performance of all ensemble methods.

Ensemble	Multilabel F1 (aggregate)		F1 per label				Binary		
	Micro	Samples	L0	L1	L2	L3	ACC	AUC	F1
E1	0.28	0.18	0.22	0.30	0.35	0.31	0.55	0.64	0.63
E2	0.29	0.18	0.20	0.32	0.40	0.32	0.46	0.55	0.63
E3	0.33	0.16	0.31	0.40	0.40	0.23	0.58	0.63	0.54
E4	0.29	0.19	0.21	0.31	0.40	0.32	0.46	0.63	0.63
E5	0.31	0.18	0.24	0.36	0.47	0.25	0.51	0.59	0.55
E6	0.26	0.20	0.22	0.29	0.27	0.24	0.56	0.66	0.59

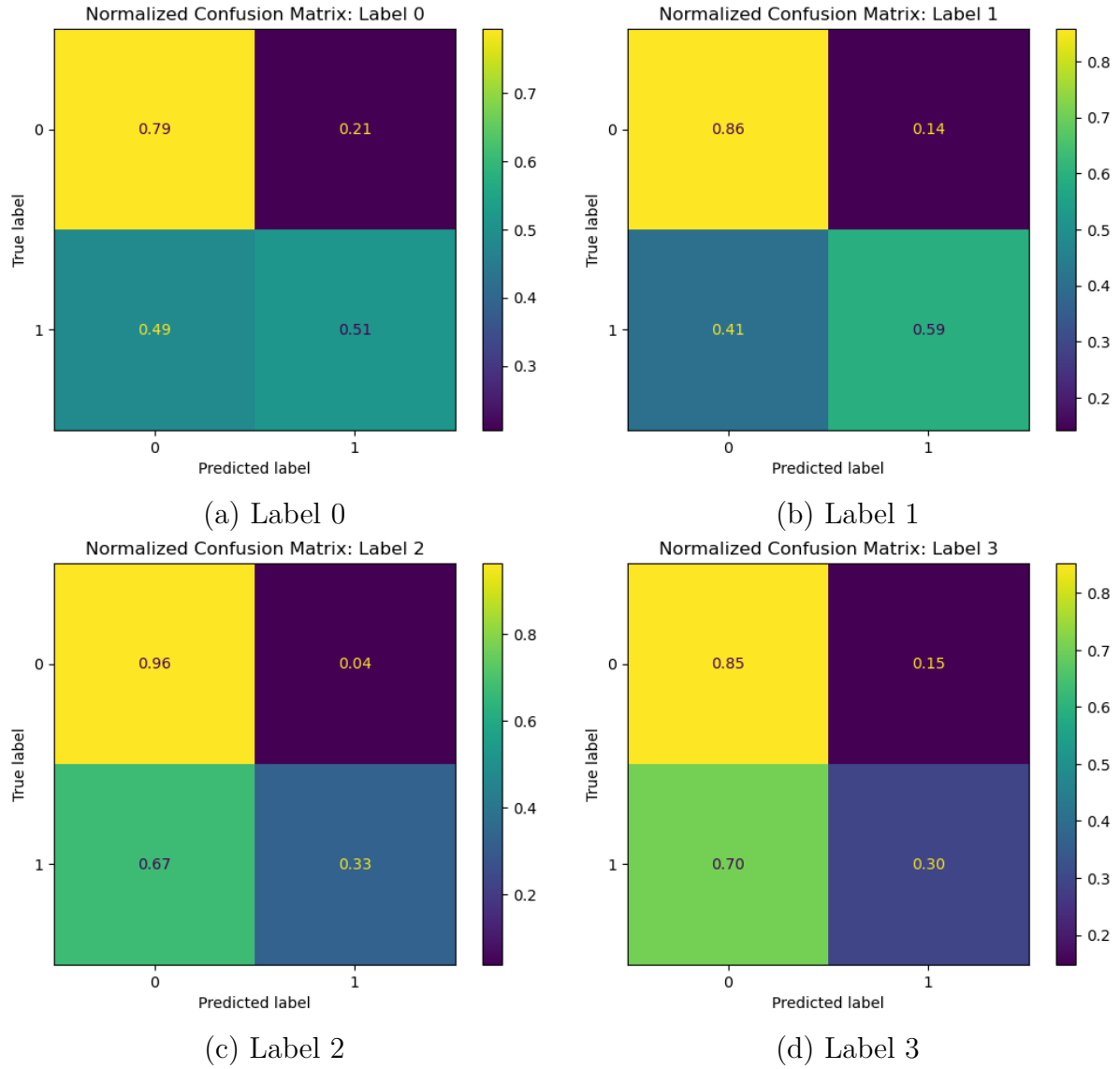


Figure 4.1: E3, per label Confusion Matrix.

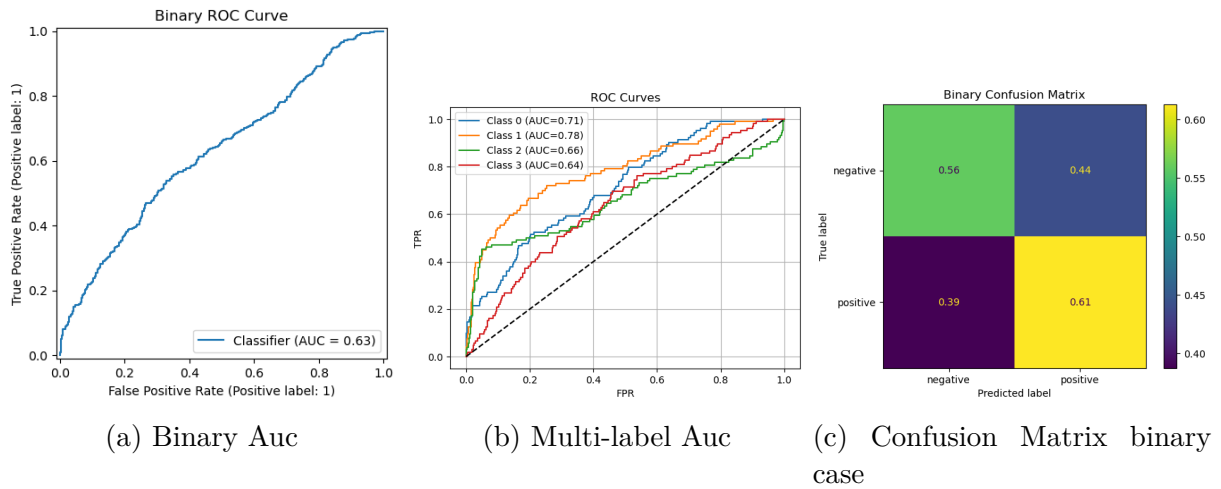


Figure 4.2: E3, other

Chapter 5

Conclusion

In this project, we explored multiple methods to detect brake faults using time-series sensor data and tabular features. We first implemented three deep-learning models (FCN, 2D CNN and LSTM) on both original and extended datasets. Then, we applied generative techniques (CGAN and RNN) to augment the data and measured their effect on a base LSTM classifier. Finally, we combined a weighted CNN and an XGBoost classifier-chain in several ensemble schemes.

Our main conclusions are:

- **Best single model:** The weighted 2D CNN on the extended dataset (“Vivid Nebula”) achieved the strongest balance between binary accuracy (0.61) and macro-F1 (0.32) as well as acceptable binary c. It handled both majority and minority classes more evenly than FCN or LSTM variants.
- **Data augmentation:** Mixing real and GAN-generated data in equal parts improved the LSTM’s F1 score from 0.36 to 0.43. However, training on fully synthetic data caused the model to collapse, indicating that GANs can interpolate existing patterns but not create new information.
- **Ensemble learning:** A gated selector blending CNN and XGBoost probabilities reached the highest ensemble performance (binary ACC = 0.58, micro-F1 = 0.33), showing that combining models with different strengths can outperform any single approach.

A critical reflection shows that severe class imbalance remained the biggest challenge. Standard techniques—dropout, class weighting, simple augmentation—were not enough to correct the skewed label distribution. While GAN-based expansion helped moderately, excessive synthetic data hurt generalisation.

Training cost was also significant: deep models took 2–4 minutes per run, making extensive hyperparameter searches and k-fold validation costly. More complex architectures (Transformers, classifier chains) gave useful insights on rare classes but did not consistently beat the weighted CNN baseline.

For future work, we recommend:

- **Advanced imbalance methods:** Try focal loss, SMOTE or adaptive thresholding to improve rare-class recall.
- **Hybrid ensembles:** Stack temporal (LSTM/Transformer) and spatial (CNN) models with attention-based gating.

- **Feature enhancement:** Incorporate frequency-domain and topological features more deeply.
- **Robust evaluation:** Use stratified k-fold cross-validation and uncertainty estimation (e.g. Monte Carlo dropout) to strengthen performance claims.

Overall, the weighted CNN on extended inputs is a strong baseline for brake-fault detection, but overcoming extreme imbalance will require more advanced data and loss strategies. This work provides a clear foundation for future safety-critical applications.

Bibliography

- [1] Henry Adams, Tegan Emerson, Michael Kirby, Rachel Neville, Chris Peterson, Patrick Shipman, Sofya Chepushtanova, Eric Hanson, Francis Motta, and Lori Ziegelmeier. Persistence images: A stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18(8):1–35, 2017.
- [2] Alexander Amini, Ava P. Soleimany, Wilko Schwarting, Sangeeta N. Bhatia, and Daniela Rus. Uncovering and mitigating algorithmic bias through learned latent structure. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society*, AIES '19, page 289–295, New York, NY, USA, 2019. Association for Computing Machinery.
- [3] Martin Arjovsky and Léon Bottou. Towards principled methods for training generative adversarial networks, 2017.
- [4] Yizhou Chen, Xu-Hua Yang, Zihan Wei, Ali Asghar Heidari, Nenggan Zheng, Zhicheng Li, Huiling Chen, Haigen Hu, Qianwei Zhou, and Qiu Guan. Generative adversarial networks in medical image augmentation: A review. *Computers in Biology and Medicine*, 144:105382, 2022.
- [5] Felisia Handayani and Metty Mustikasari. Sentiment analysis of electric cars using recurrent neural network method in indonesian tweets. *Jurnal Ilmiah Kursor*, 10, 12 2020.
- [6] Pankaj Malhotra, Lovekesh Vig, Gautam Shroff, and Puneet Agarwal. Long short term memory networks for anomaly detection in time series. In *European Symposium on Artificial Neural Networks (ESANN)*, 2016.
- [7] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets, 2014.
- [8] Francisco Javier Ordóñez and Daniel Roggen. Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition. *Sensors*, 16(1):115, 2016.
- [9] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget, 2024.
- [10] Huaxiu Yao, Feixiang Wu, Junbo Ke, Xianfeng Tang, Yitian Jia, Shuming Lu, and Zhenhui Li. Deep multi-view spatial-temporal network for taxi demand prediction. In *AAAI Conference on Artificial Intelligence*, 2018.